OFFSIDE LABS

DLMM 0.12.0

Smart Contract Security Assessment

March 2026

Prepared for:

Meteora

Prepared by:

Offside Labs

Sirius Xie

Ronny Xing

Contents

1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.

 **https://offside.io/**

 **https://github.com/offsidelabs**

 **https://twitter.com/offside_labs**

2 Executive Summary

Introduction

Offside Labs completed a security audit of *DLMM* smart contracts, starting on February 3rd, 2026, and concluding on March 5th, 2026.

Project Overview

DLMM Release 0.12.0 adds limit orders and integrates them into swap execution so swaps can match against both bin market making liquidity and limit order liquidity. It refactors swap and swap2 to use a shared swap engine, updates bin level swap math and accounting, and introduces configurable fee collection mode that changes where fees are taken and how protocol fees are accumulated. It also extends pair state and related loaders to support the new limit order lifecycle and updated swap processing across bin arrays.

Audit Scope

The assessment scope contains mainly the smart contracts of the lb-clmm program for the *DLMM* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- DLMM
  - Codebase: https://github.com/MeteoraAg/DLMM
  - Release 0.12.0: PR-466
  - Commit Hash: 97069565eca24b544d7809582f45acfcc5869384

The issues described in this report have been resolved by the following commit:

- DLMM
  - Codebase: https://github.com/MeteoraAg/DLMM
  - Commit Hash: c1aa2148389da8a607585eeea808b99f0fd56b3b

We listed the files we have audited below:

- DLMM
  - programs/lb_clmm/src/constants.rs
  - programs/lb_clmm/src/errors.rs
  - programs/lb_clmm/src/events.rs
  - programs/lb_clmm/src/fee_calculator.rs
  - programs/lb_clmm/src/instructions/admin/close_bin_array.rs
  - programs/lb_clmm/src/instructions/claim_fee.rs
  - programs/lb_clmm/src/instructions/claim_reward.rs
  - programs/lb_clmm/src/instructions/deposit/add_liquidity.rs
  - programs/lb_clmm/src/instructions/deposit/add_liquidity_by_strategy.rs
  - programs/lb_clmm/src/instructions/deposit/add_liquidity_by_weight_one_side.rs

- programs/lb_clmm/src/instructions/initialize_pool/initialize_customizable_permissionless_lb_pair.rs
- programs/lb_clmm/src/instructions/initialize_pool/initialize_permission_lb_pair.rs
- programs/lb_clmm/src/instructions/limit_order/cancel_limit_order.rs
- programs/lb_clmm/src/instructions/limit_order/close_limit_order_if_empty.rs
- programs/lb_clmm/src/instructions/limit_order/place_limit_order.rs
- programs/lb_clmm/src/instructions/operator/fund_reward.rs
- programs/lb_clmm/src/instructions/operator/initialize_preset_parameters.rs
- programs/lb_clmm/src/instructions/operator/initialize_reward.rs
- programs/lb_clmm/src/instructions/operator/update_reward_duration.rs
- programs/lb_clmm/src/instructions/operator/withdraw_ineligible_reward.rs
- programs/lb_clmm/src/instructions/permissionless_operations/go_to_a_bin.rs
- programs/lb_clmm/src/instructions/permissionless_operations/increase_oracle_length.rs
- programs/lb_clmm/src/instructions/position/decrease_position_length.rs
- programs/lb_clmm/src/instructions/position/set_permissionless_operation_bits.rs
- programs/lb_clmm/src/instructions/position_authorize.rs
- programs/lb_clmm/src/instructions/rebalance/rebalance_params.rs
- programs/lb_clmm/src/instructions/rebalance/rebalance_wrapper.rs
- programs/lb_clmm/src/instructions/swap.rs
- programs/lb_clmm/src/instructions/update_fees_and_rewards.rs
- programs/lb_clmm/src/instructions/v2/add_liquidity2.rs
- programs/lb_clmm/src/instructions/v2/add_liquidity_by_strategy2.rs
- programs/lb_clmm/src/instructions/v2/add_liquidity_by_weight2.rs
- programs/lb_clmm/src/instructions/v2/add_liquidity_single_side_precise2.rs
- programs/lb_clmm/src/instructions/v2/claim_fee2.rs
- programs/lb_clmm/src/instructions/v2/claim_reward2.rs
- programs/lb_clmm/src/instructions/v2/remove_liquidity2.rs
- programs/lb_clmm/src/instructions/v2/swap2.rs
- programs/lb_clmm/src/instructions/v2/update_fees_and_reward2.rs
- programs/lb_clmm/src/instructions/withdraw/remove_all_liquidity.rs
- programs/lb_clmm/src/instructions/withdraw/remove_liquidity.rs
- programs/lb_clmm/src/lb_access_control.rs
- programs/lb_clmm/src/lib.rs
- programs/lb_clmm/src/math/safe_math.rs
- programs/lb_clmm/src/math/utils_math.rs
- programs/lb_clmm/src/math/weight_to_amounts.rs
- programs/lb_clmm/src/state/bin.rs
- programs/lb_clmm/src/state/dynamic_position.rs
- programs/lb_clmm/src/state/lb_pair.rs
- programs/lb_clmm/src/state/limit_order.rs
- programs/lb_clmm/src/state/operator.rs
- programs/lb_clmm/src/state/parameters.rs
- programs/lb_clmm/src/state/position.rs

- programs/lb_clmm/src/state/preset_parameters.rs
- programs/lb_clmm/src/state/preset_parameters2.rs
- programs/lb_clmm/src/utils/token2022.rs
- programs/lb_clmm/src/utils/transfer_memo.rs

Findings

The security audit revealed:

- 1 critical issues
- 1 high issues
- 0 medium issue
- 2 low issues
- 4 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.

3 Summary of Findings

ID	Title	Severity	Status
01	Missing Token-2022 TransferFee in exact_in mode in swap2 IXs	Critical	Fixed
02	Incorrect Reset May Prevent Cancellation of bin_-id == 0 Order	High	Fixed
03	Inconsistent Token-2022 TransferFee Handling in add_liquidity_by_weight2 IX	Low	Fixed
04	close_bin_array May Close BinArrays With Pending Filled Limit Order	Low	Fixed
05	Missing support_liquidity_mining checks	Informational	Fixed
06	Processed Orders Can Be Fully Filled While Still Classified as PartialFilled	Informational	Acknowledged
07	Potential Compatibility Risk From Reused Bin Tombstone Bytes Under Undetermined LbPair	Informational	Acknowledged
08	Legacy Bin.price May be 0	Informational	Acknowledged

4 Key Findings and Recommendations

4.1 Missing Token-2022 TransferFee in exact_in mode in swap2 IXs

Severity: Critical	Status: Fixed
Target: Smart Contract	Category: Token

Description

At the end of the `swap2` instructions' internal method `internal_handle_swap2`, it calls `Swap2.perform_swap` to execute transfers between the user and the LbPair based on the computed result.

In `exact in` mode, the `amount_in` argument passed to `Swap2.perform_swap` is `swap_final_result.user_pay_amount`, which has the Token 2022 TransferFee deducted.

```
408     if exact_amount_out.is_none() {
409         // swap exact in
410         ctx.accounts.perform_swap(
411             swap_for_y,
412             fee_mode.fees_on_input,
413             swap_final_result.host_fee,
414             swap_final_result.user_pay_amount,
415             swap_final_result.user_receive_amount,
416             transfer_hook_in_accounts,
417             transfer_hook_out_accounts,
418         )?;
419     } else {
```

programs/lb_clmm/src/instructions/v2/swap2.rs#L408-L419

Then, in `Swap2.perform_swap`, only `swap_final_result.user_pay_amount` of the input mint is transferred from the user's account.

```
119     fn perform_swap(
120         ...
121         amount_host_fee: u64,
122         amount_in: u64,
123         amount_out: u64,
124         ...
125     ) -> Result<()> {
126                 ...
127         // trasfer from user to pool for total amount in
128         transfer_from_user2(
129             &self.user,
130             token_mint_in,
131             &self.user_token_in,
132             &token_vault_in.to_account_info(),
133             token_program_in,
134             amount_in,
135             None,
136             transfer_hook_in_accounts,
137         )?;
```

programs/lb_clmm/src/instructions/v2/swap2.rs#L119-L148

Impact

When the input mint has the Token 2022 TransferFee extension enabled and the fee rate is greater than zero, the user can swap out an amount of output mint corresponding to the pre transfer fee input amount while only providing a smaller input amount after the transfer fee is deducted, which directly causes losses to the LPs in the LbPair.

Recommendation

It's recommended that in exact in mode, the original `amount_in` is passed directly, or follow the exact out branch by using `calculate_transfer_fee_included_amount` to derive the transfer fee included amount and assert that this amount equals the user provided `amount_in` .

Mitigation Review Log

Fixed in the commit 7673dccc41e82bc3dbb50935555ba30e937d06cd.

4.2 Incorrect Reset May Prevent Cancellation of bin_id == 0 Order

Severity: High	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

In the `cancel_limit_order` instruction, the canceled LimitOrderBinData is reset, and during this reset, `LimitOrderBinData.bin_id` is also reset to 0.

```rust
    pub fn cancel_order(&mut self) {
        // set all fields to default
        self.amount = 0;
        self.age = 0;
        self.bin_id = 0;
        self.is_ask = 0;
    }
```

programs/lb_clmm/src/state/limit_order.rs#L22-L28

Suppose there are two uncanceled LimitOrderBinData entries at indices `x` and `y` in `DynamicLimitOrder.limit_order_bins`, and they satisfy:

- x < y
- limit_order_bins[y].bin_id == 0

If the user first cancels `limit_order_bins[x]`, then `limit_order_bins[x].bin_id` is set to `0`. After that, it becomes impossible to cancel `limit_order_bins[y]`, because during cancellation, `DynamicLimitOrder.get_limit_order_bin_data_mut_checked` will always retrieve `limit_order_bins[x]` first.

```rust
    pub fn get_limit_order_bin_data_mut_checked(
        &mut self,
        bin_id: i32,
    ) -> Result<&mut LimitOrderBinData> {
        for limit_order in self.limit_order_bins.iter_mut() {
            if limit_order.bin_id == bin_id {
                return Ok(limit_order);
            }
        }
```

programs/lb_clmm/src/state/limit_order.rs#L132-L140

Impact

This incorrect reset may cause the LimitOrder user to be permanently unable to cancel a LimitOrder placed at `bin_id == 0`, which could prevent the user from withdrawing the funds locked in the LimitOrder on bin 0.

Recommendation

It's recommended not to reset the bin_id field in `LimitOrderBinData.cancel_order`, or to have `get_limit_order_bin_data_mut_checked` skip LimitOrderBinData entries where `limit_order.is_empty() == true`.

Mitigation Review Log

Fixed in the commit 30d68a196bb393adc27ea676587d15e4398cc16f.

4.3 Inconsistent Token-2022 TransferFee Handling in add_liquidity_by_weight2 IX

Severity: Low	Status: Fixed
Target: Smart Contract	Category: Token

Description

In the v2 `add_liquidity2` and `add_liquidity_by_strategy2` instructions, the user-provided amount_x/amount_y are first reduced by the Token2022 transfer fee to obtain the transfer-fee-excluded amounts, which are then used for subsequent calculations and accounting. Finally, the net amounts deposited into the pool are converted back to transfer-fee-included amounts and transferred out from the user's accounts.

However, in the newly added `add_liquidity_by_weight2` instruction, `LiquidityParameterByWeight.amount_x` and `amount_y` are used directly to compute `amounts_in_bin` and then passed into `handle_deposit_by_amounts2`, without first calling `calculate_transfer_fee_excluded_amount` as the other v2 add_liquidity2_* instructions do.

```
19          let (amount_x, amount_y) = if liquidity_parameter.is_include_bin_id(active_id) {
20              let transfer_hooks_account_len: usize = remaining_accounts_info.total_length().into();
21              if transfer_hooks_account_len > ctx.remaining_accounts.len() {
22                  return Err(LBError::InsufficientRemainingAccounts.into());
23              }
24              let bin_arrays_accounts = &mut &ctx.remaining_accounts[transfer_hooks_account_len..];
25              find_amount_in_active_bin(ctx.accounts.lb_pair.key(), active_id, bin_arrays_accounts)?
26          } else {
27              (0, 0)
28          };
29          liquidity_parameter.to_amounts_into_bin(active_id, bin_step, amount_x, amount_y)?
```

programs/lb_clmm/src/instructions/v2/add_liquidity_by_weight2.rs#L19-L29

This means that under the Token2022 TransferFee scenario, the parameter semantics of `add_liquidity_by_weight2` are inconsistent with the other v2 `add_liquidity2_*` instructions: it treats the parameters as transfer-fee-excluded amounts for allocation,

while `handle_deposit_by_amounts2` then derives transfer-fee-included amounts from those transfer-fee-excluded amounts and transfers them from the user's accounts.

Impact

This inconsistency may cause the actual amount transferred from the user to be greater than the amount specified in the instruction parameters.

Recommendation

It's recommended to unify the parameter semantics across all `v2 add_liquidity2_*` instructions, keeping `add_liquidity_by_weight2` consistent with the other v2 instructions , i.e., add_liquidity2, add_liquidity_by_strategy2.

Mitigation Review Log

Fixed in the commit 2503e37d3478a0007a0d20c9aec6976d64de654c.

4.4 close_bin_array May Close BinArrays With Pending Filled Limit Order

Severity: Low	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

For limit order enabled pools, `Bin.is_zero_liquidity(true)` and `BinArray.is_zero_liquidity(true)` only consider `liquidity_supply` , `open_order_amount` , and `processed_order_remaining_amount` , but do not consider `fulfilled_order_amount_x` and `fulfilled_order_amount_y` .

```
285     pub fn is_zero_liquidity(&self, is_support_limit_order: bool) -> bool
        ↪ {
286         if is_support_limit_order {
287             self.liquidity_supply == 0
288                 && self.open_order_amount == 0
289                 && self.processed_order_remaining_amount == 0
290         } else {
291             self.liquidity_supply == 0
292         }
293     }
```

programs/lb_clmm/src/state/bin.rs#L285-L293

As a result, in `close_bin_array` instruction, a bin that has no LP liquidity and no pending limit orders, but still has non zero fulfilled limit order amounts that are pending maker

withdrawal, can be treated as zero liquidity.

```
36      // Extra safety check
37      for bin in bin_array.bins.iter() {
38          require!(
39              bin.is_zero_liquidity(lb_pair.support_limit_order()?),
40              LBError::UndeterminedError
41          );
42      }
```

programs/lb_clmm/src/instructions/admin/close_bin_array.rs#L36-L42

Impact

Although this instruction can only be invoked by an admin, and such a BinArray account is unlikely to be created in the current version, such a BinArray may still be treated as eligible for closure and be closed. If that happens, makers may no longer be able to cancel or collect the corresponding fulfilled limit order amounts for bins stored in the closed bin array, causing funds to be stuck.

Recommendation

It's recommended to add an explicit non zero fulfilled amount check in `close_bin_array` before allowing closure.

Mitigation Review Log

Fixed in the commit 578ac20bc93ea4c2c4a7ed58485fc827eb59fed1.

4.5 Informational and Undetermined Issues

Missing support_liquidity_mining checks

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Data Validation

`swap2` currently checks whether to run reward distribution using `lb_pair.has_reward()` rather than `lb_pair.support_liquidity_mining()`. `has_reward()` only checks whether any `reward_infos[i]` is initialized and does not enforce that the pool is actually operating in `LiquidityMining` mode. If reward_infos contains non default residual values on a pool that is effectively treated as LimitOrder mode, the reward distribution path may still execute and write per bin reward per token stored state, which may corrupt limit order state and lead to inconsistent state updates.

`update_reward_funder` does not enforce `lb_pair.support_liquidity_mining()` like `update_reward_duration` does, and only checks that the target reward_info is initialized.

If a pool is not intended to be LiquidityMining but ends up with an initialized reward_info due to stale bytes, this instruction can still mutate reward configuration on that pool.

It's recommended to add an explicit `lb_pair.support_liquidity_mining()` check in `swap2` and `update_reward_funder`.

Processed Orders Can Be Fully Filled While Still Classified as PartialFilled

Severity: Informational	Status: Acknowledged
Target: Smart Contract	Category: Logic Error

In `get_cancel_limit_order_result`, there is an edge case condition: when the most recent swap only takes the branch that processes the processed order and makes `Bin.processed_order_remaining_amount == 0`, `Bin.order_age` may still not be advanced because that path does not update `Bin.order_age`. In this case, there is no remaining processed amount.

```
666         } else if
            ↪ swap_result.filled_amount_result.processed_order_amount_out >
            ↪ 0 {
667             // update limit order liquidity
668             self.processed_order_remaining_amount = self
669                 .processed_order_remaining_amount
670
    ↪ .safe_sub(swap_result.filled_amount_result.processed_order_amount_out)?;
671         }
```

programs/lb_clmm/src/state/bin.rs#L666-L671

However, `get_limit_order_status` determines the status based on the difference between `LimitOrderBinData.age` and `Bin.order_age`, so it may still be classified as `PartialFilled`. Then `get_cancel_limit_order_result` will enter the `PartialFilled` branch, and `Bin.apply_cancel_limit_order` will continue to update `Bin.total_processing_order_amount`.

Although updating this field has limited impact, it's recommended to add stricter constraints based on remaining/processing amounts in the classification logic of `get_limit_order_status` / `get_cancel_limit_order_result` (e.g., when the conditions for updating `order_age` are met, treat `processed_order_remaining_amount == 0` as `Fulfilled`), and ensure the delta updates in the `PartialFilled` and `Fulfilled` branches do not modify fields that should not be modified.

Potential Compatibility Risk From Reused Bin Tombstone Bytes Under Undetermined LbPair

Severity: Informational	Status: Acknowledged
Target: Smart Contract	Category: Compatibility

In the current release, `LbPair.support_limit_order()` may treat a pair as supporting limit orders when `function_type` is `Undetermined` and no rewards are initialized. The Bin account layout historically used the same byte regions for `function_bytes` and fields such as `amount_y_in`, and the current version reuses those regions for limit order related state fields (e.g., limit_order_ask_side).

If any historical `Bin` accounts contain non zero residual data in these reused regions, the new logic may misinterpret those bytes as valid limit order state, which can affect swap and cancel flows, potentially leading to unexpected behavior, such as instruction failures and inconsistent state updates.

It's recommended to confirm and validate that all historical LbPair and Bin accounts guarantee zeroed values in these reused tombstone regions.

Legacy Bin.price May be 0

Severity: Informational	Status: Acknowledged
Target: Smart Contract	Category: Compatibility

In this release, the `Bin.get_or_store_bin_price` method has been removed, meaning it no longer supports dynamically computing the current bin price at runtime. If any legacy bins still have an uninitialized `Bin.price` of 0, then adding liquidity would use 0 as the price, which is unintended. It's recommended to confirm, before deploying this Release, that all existing bins have their `Bin.price` properly initialized.

5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

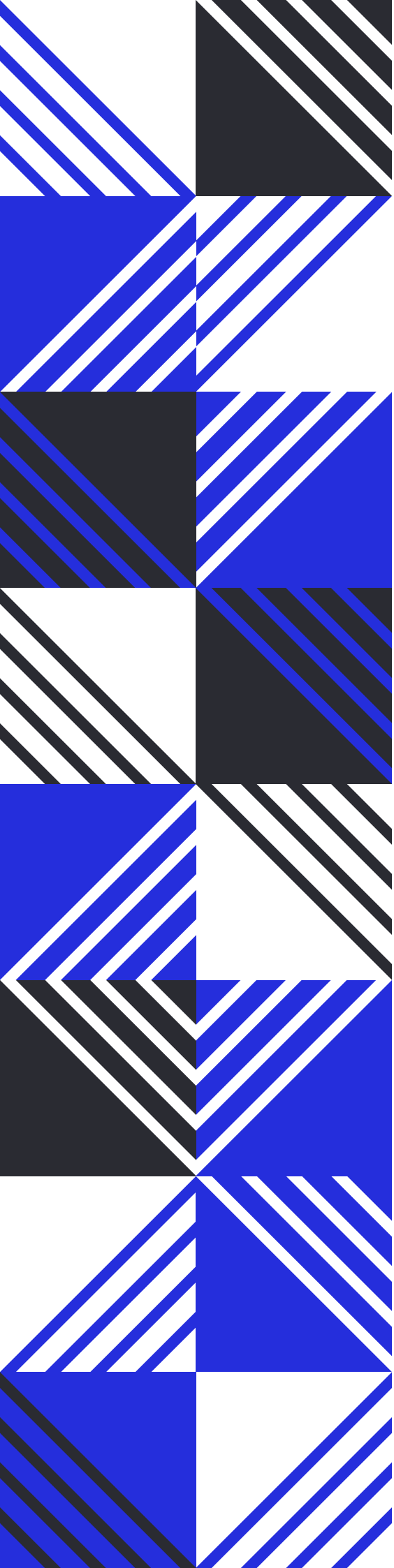
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



OFFSIDE LABS

https://offside.io/

https://github.com/offsidelabs

https://twitter.com/offside_labs